Graphs II

Distance, Constraints, and Logical Structure
Dijkstra and 0–1 BFS

KCPC

In collaboration with KCL Tech
2025–2026 partnership

Week 5

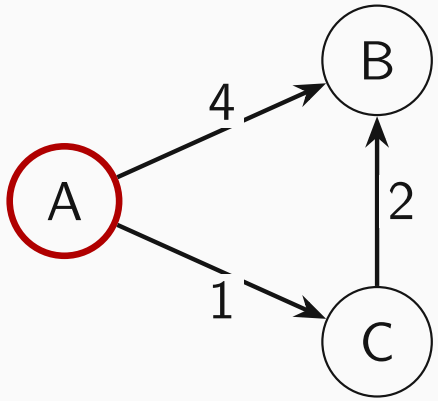
Today's route

1. Model a graph; recall DFS, BFS and multi-source BFS.
2. Find minimum total cost with **Dijkstra**.
3. Specialise to binary costs with **0–1 BFS**.
4. Translate grids and path variants into the right model.

Live examples: problems **1–3**. Then try the core set.

Checkpoints **C1–C6** have full answers *after the conclusion*.

A graph is a set of states and moves



- → **Vertex**: a state, such as a city or grid cell.
- → **Edge**: a legal move between states.
- → **Weight**: the cost of that move.
- → Arrows mean **directed**: the reverse move is not automatic.

A path follows edges. Its usual cost is the **sum of edge weights**.

Store neighbours in an adjacency list

For the previous directed graph:

```
adj = {
    "A": [("B", 4), ("C", 1)],
    "B": [],
    "C": [("B", 2)]
}
```

- → `adj[u]` contains pairs `(neighbour, weight)`.
- → For an **undirected** edge, insert *both* directions.
- → Usually number vertices $0, \ldots, V-1$ and use a list of lists.
- → V vertices and E edges take $O(V+E)$ storage.

DFS reaches; BFS finds fewest edges

- → **DFS**: follow a branch, then backtrack using a stack.
  Good for reachability; the first path found need not be shortest.
- → **BFS**: use a FIFO (first in, first out) queue, visiting one layer at a time.
  A vertex first reached from layer k has distance $k + 1$.
- → In ordinary BFS, mark a vertex when **enqueuing** it.
  This avoids repeatedly adding the same vertex.

BFS minimises **edge count**: also minimum cost when every edge has the same positive cost. Both traversals take $O(V + E)$ time.

Multi-source BFS: nearest of many starts

Problem 7: LC542, 01 Matrix. Distance to the nearest zero.

$$\begin{pmatrix} 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 0 \end{pmatrix} \longrightarrow \begin{pmatrix} 0 & 1 & 2 \\ 1 & 2 & 1 \\ 2 & 1 & 0 \end{pmatrix}$$

1. Set every zero's distance to 0 and put **all zeros** in one queue.
2. Other cells start at ∞; expand four-direction neighbours.
3. On first discovery, set $\mathrm{dist}[v] = \mathrm{dist}[u] + 1$.

The first wave to reach a cell comes from its nearest source. One BFS, $O(RC)$ time; no separate search for each zero.

Try yourself C1: nearest zero

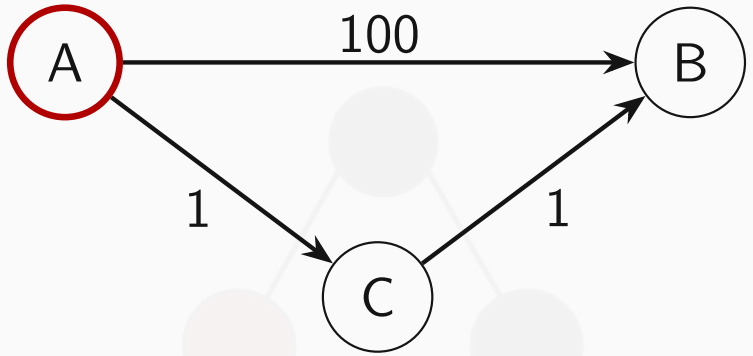
$$\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix}$$

You may move up, down, left or right within the grid.

1. Which cells enter the queue initially, and at what distance?
2. Compute the complete distance matrix.
3. Why would a DFS's first-found route not justify these distances?

Coordinates in this deck are **zero-based**: top left is $(0, 0)$.

Fewest edges can be expensive



$A \to B:\ 100$ $\qquad$ $A \to C \to B:\ 1 + 1 = 2$

Ordinary BFS prefers one edge. We need to prefer **lower total cost**.

Dijkstra handles **nonnegative weights**, including zero.

Relaxation: improve a best-known route

- → $\mathrm{dist}[v]$ is the cheapest route **found so far** to v.
- → Initially $\mathrm{dist}[s] = 0$; everything else is ∞.
- → From u, try each edge $u \to v$ with weight w:

$$\mathrm{candidate} = \mathrm{dist}[u] + w.$$

If $\mathrm{candidate} < \mathrm{dist}[v]$, replace the label and queue it. This improvement is called **relaxing** the edge.

Which queued route next? Use a **min-heap**: it removes the entry with the smallest distance, not the oldest entry.

The invariant that makes Dijkstra safe

Invariant: when a *current* minimum-distance entry for u is removed, its distance is the true shortest distance.

- → Why? A cheaper route would have to pass through an unprocessed vertex with a smaller distance first.
- → That vertex would have been processed earlier. Nonnegative edges cannot make a later route suddenly cheaper.
- → Hence u is **settled**: its label will not improve again.

A negative edge breaks this argument. Do **not** use Dijkstra for general negative-weight graphs.

Dijkstra in Python: lazy min-heap

Here `adj[u]` is a list of `(v, w)` pairs; vertices are integers.

```
from heapq import heappop, heappush

def dijkstra(adj, src):
    dist = [float("inf")] * len(adj)
    dist[src] = 0
    heap = [(0, src)]
    while heap:
        d, u = heappop(heap)
        if d != dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heappush(heap, (nd, v))
    return dist
```

Stale entries and the right time to stop

- → Heap entries are **snapshots** `(distance, vertex)`. Improving a label adds a new entry; the old one stays in the heap.
- → If popped $d \neq \mathrm{dist}[u]$, the snapshot is **stale**. Skip it without scanning neighbours.
- → **One target?** After the stale check, stop when the target is *popped*. Its distance is now final.
- → **All distances?** Keep going until the heap is empty.

Do not stop on first discovery or mark a vertex final on insertion.

Live 1 / problem 1: GFG Dijkstra

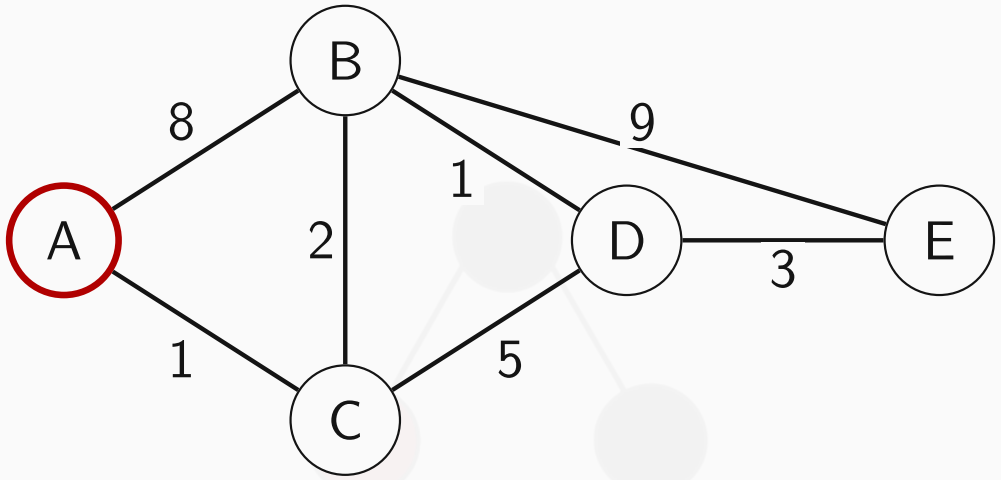
GFG: Dijkstra Algorithm currently supplies an **undirected edge list**, not a matrix.

```
class Solution:
    def dijkstra(self, V, edges, src):
        adj = [[] for _ in range(V)]
        for u, v, w in edges:
            adj[u].append((v, w))
            adj[v].append((u, w))
        return dijkstra(adj, src)
```

This wrapper calls the previous helper. Return distances in vertex order.

The historical URL still says `adjacency-matrix`; use the **current platform signature** `V, edges, src`.

Live 1: a graph we can trace exactly



Undirected. Source A; target E. Labels A–E map to integers 0–4.

$$\text{edges} = [(0,1,8),(0,2,1),(1,2,2),(1,3,1),$$
$$(2,3,5),(1,4,9),(3,4,3)].$$

Predict the order of **valid pops**. An edge's weight is not necessarily the distance of its endpoint.

Live 1: reproducible heap trace

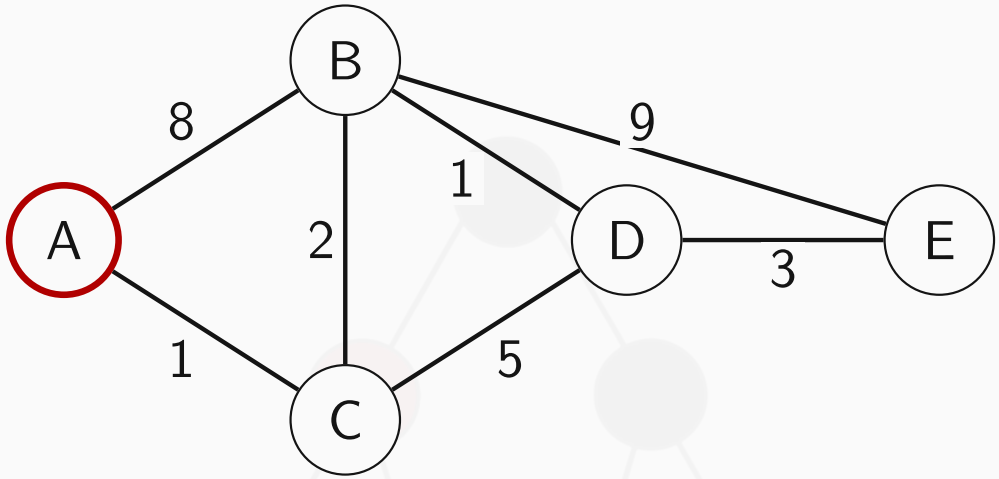
Entries below are **(distance, vertex)**, displayed in sorted order.
A binary heap need not store them in this visual order internally.

Pop	Improved labels	Pending entries after the pop
$(0, \mathrm{A})$	B: 8, C: 1	$(1, \mathrm{C}), (8, \mathrm{B})$
$(1, \mathrm{C})$	B: 3, D: 6	$(3, \mathrm{B}), (6, \mathrm{D}), (8, \mathrm{B})$
$(3, \mathrm{B})$	D: 4, E: 12	$(4, \mathrm{D}), (6, \mathrm{D}), (8, \mathrm{B}), (12, \mathrm{E})$
$(4, \mathrm{D})$	E: 7	$(6, \mathrm{D}), (7, \mathrm{E}), (8, \mathrm{B}), (12, \mathrm{E})$
$(6, \mathrm{D})$	stale: skip	$(7, \mathrm{E}), (8, \mathrm{B}), (12, \mathrm{E})$
$(7, \mathrm{E})$	none	$(8, \mathrm{B}), (12, \mathrm{E})$
$(8, \mathrm{B})$	stale: skip	$(12, \mathrm{E})$
$(12, \mathrm{E})$	stale: skip	empty

Final distances: $[0, 3, 1, 4, 7]$. Shortest route to E:
$\mathrm{A} \to \mathrm{C} \to \mathrm{B} \to \mathrm{D} \to \mathrm{E}$.

For E only, stop at **(7, E)**, not when E is first labelled 12.

Try yourself C2: a free final edge

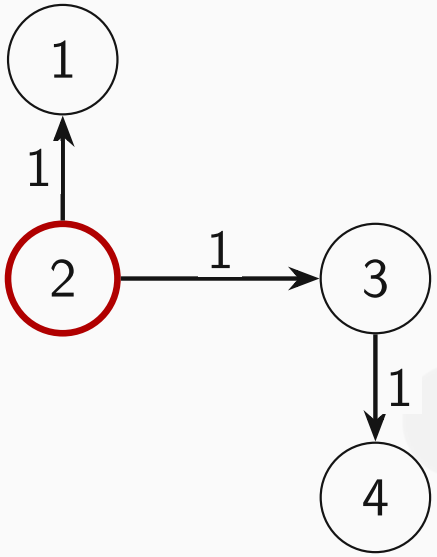


Change **only D–E from weight 3 to weight 0**.

1. Find the new final distances from A and a shortest route to E.
2. When is E first discovered? When is it safe to stop?
3. List the valid pops, breaking equal-distance ties alphabetically.

Live 2 / problem 2: Network Delay Time

LC743: a signal starts at vertex k. When has *every* vertex received it? Edges are **directed** travel times.



$n = 4,\ k = 2.$
Input: $(2, 1, 1)$, $(2, 3, 1)$, $(3, 4, 1)$.
Distances in order 1–4:
$[1, 0, 1, 2]$.
Return $\max_v \mathrm{dist}[v] = \boxed{2}$.

If any vertex is unreachable, return -1. With a size $n + 1$ array, ignore unused index 0 in the answer.

Dijkstra: implementation checklist

- → All weights must be **nonnegative**; zero-cost edges are fine.
- → Relax only on a **strict improvement**; stale pops do no work.
- → Unreachable vertices stay at ∞. Use sufficiently wide integers in C++ for sums of many weights.
- → To recover a route, set `parent[v] = u` on improvement; follow parents backwards from the target.

Lazy heap: $O(V + E \log(E + 1))$ time, $O(V + E)$ space. For a simple graph, the usual time bound is $O((V + E) \log V)$.

Binary costs do not need a full heap

Suppose every edge weight is **0 or 1**.

- → From a route of cost d, the next cost is only d or $d + 1$.
- → A **deque** lets us insert at either end and pop from the front.
- → A weight-0 improvement goes to the **front**: same cost layer.
- → A weight-1 improvement goes to the **back**: next cost layer.

This is **0–1 BFS**: Dijkstra's ordering, without a heap. Ordinary FIFO BFS would count edges, not total cost.

The deque invariant: compare stored keys

Store snapshots (d, u), just as in the heap.

$$\underbrace{d,\ d,\ d}_{\text{front level}} \quad \underbrace{d+1,\ d+1}_{\text{next level}}$$

→ Stored distance keys are nondecreasing, with at most **two adjacent levels**.

→ On a current pop of cost d, front-inserting d or back-inserting $d+1$ preserves this order.

→ Labels may improve while old snapshots remain. Skip a pop when $d \neq \mathrm{dist}[u]$.

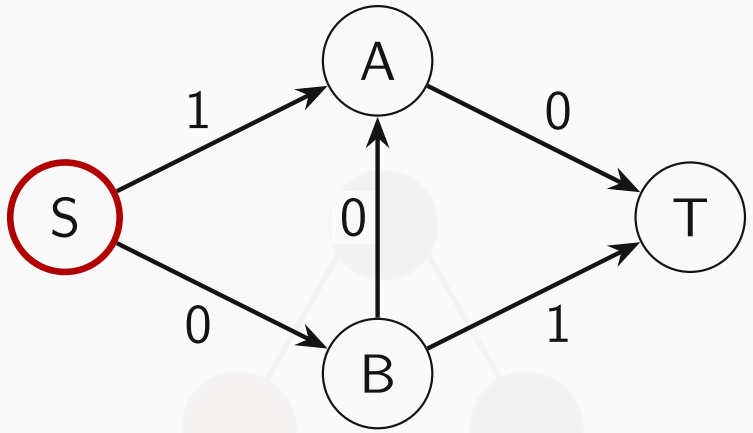
The current front is minimum, so the same settling argument applies. Do not confuse *stored keys* with changing `dist` labels.

0–1 BFS in Python

```
from collections import deque

def zero_one_bfs(adj, src):
    dist = [float("inf")] * len(adj)
    dist[src] = 0
    q = deque([(0, src)])
    while q:
        d, u = q.popleft()
        if d != dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                if w == 0:
                    q.appendleft((nd, v))
                else:
                    q.append((nd, v))
    return dist
```

Live: a zero-cost detour beats discovery



Directed. Scan outgoing edges in exactly this order:

$$S : [(A, 1), (B, 0)] \quad A : [(T, 0)]$$
$$B : [(A, 0), (T, 1)] \quad T : []$$

A is first discovered at 1, but $S \to B \to A$ costs 0. Predict what happens to the *old* entry for A.

Live: exact deque trace, front on the left

Start with $[(0, S)]$. Each entry is a stored **(distance, vertex)**.

Pop	Improved labels	Deque after the pop
$(0, S)$	A: 1, B: 0	$[(0, B), (1, A)]$
$(0, B)$	A: 0, T: 1	$[(0, A), (1, A), (1, T)]$
$(0, A)$	T: 0	$[(0, T), (1, A), (1, T)]$
$(0, T)$	none	$[(1, A), (1, T)]$
$(1, A)$	stale: skip	$[(1, T)]$
$(1, T)$	stale: skip	empty

All four final distances are 0. A shortest route to T is $S \to B \to A \to T$.

We could stop at the **current** pop $(0, T)$, after the stale check.

Try yourself C3: two tempting mistakes

Use the previous directed graph and the same edge-scan order.

1. Suppose we mark a vertex visited *when enqueuing*, and never allow it to improve. What wrong distance reaches T?
2. Keep the correct distance updates, but change the stale test to always process old entries. What work is unnecessarily repeated?
3. Why must the relaxation test use $<$, not $\leq$, if a zero-cost cycle exists?

Explain the role of **distance improvements**, not just queue order.

0–1 BFS: what the guarantee depends on

- → Weights really must lie in $\{0, 1\}$, not just be integers.
- → Never make first *discovery* final. A zero-cost detour may improve an already queued vertex.
- → A current pop is settled; stale snapshots do not rescan edges.
- → Strict improvements also prevent endless requeuing around zero-cost cycles.

$O(V + E)$ time: each vertex's outgoing edges are scanned once on its current pop; each successful relaxation adds one snapshot.

$O(V + E)$ space including adjacency lists and queued snapshots.

A grid is an implicit graph

- → **State**: a cell (r, c); **edge**: one legal move.
- → Generate neighbours when needed: usually $(r \pm 1, c)$ and $(r, c \pm 1)$.
- → Reject out-of-bounds cells and any forbidden destinations.
- → Define the **cost of the move**, then select the algorithm.

No need to build an adjacency list for every cell. An $R \times C$ four-neighbour grid has RC vertices and $O(RC)$ edges.

A rectangular drawing does *not* imply unweighted BFS.

Live 3 / problem 3: arrow-grid costs

LC1368: change as few cell arrows as possible to obtain a path from top left to bottom right.

start

→	←	↓
→	→	↑

target

Every in-bounds orthogonal move is allowed.

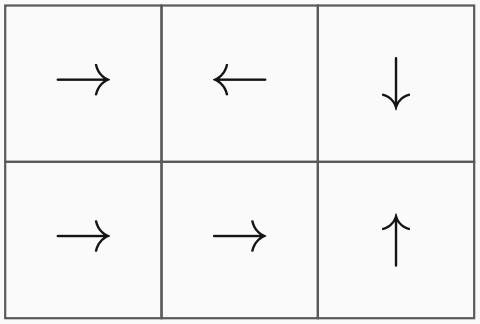
$$w(u,v) = \begin{cases} 0 & \text{follow the arrow at } u, \\ 1 & \text{otherwise.} \end{cases}$$

The arrow belongs to the cell we **leave**, not the one we enter.

Platform encoding: $1 = \rightarrow$, $2 = \leftarrow$, $3 = \downarrow$, $4 = \uparrow$. Use **0–1 BFS**.

Try yourself C4 / live 3: escape the arrow loop

start



target

Input: $\begin{pmatrix} 1 & 2 & 3 \\ 1 & 1 & 4 \end{pmatrix}$.

1. What cells can we reach at cost 0 from $(0, 0)$?
2. Find a minimum-cost route to $(1, 2)$.
3. Give a lower bound proving that your route is optimal.

Following arrows forever can loop. A shortest route can be chosen without cycles, so paying for a departure corresponds to changing that cell's arrow once.

Sometimes the cost belongs to the destination

Problem 6: LC2290, obstacle removal.

$$w\big((r, c), (r', c')\big) = \mathrm{grid}[r'][c'] \in \{0, 1\}.$$

→ Entering an obstacle costs 1; entering an empty cell costs 0. Use **0–1 BFS**. The source is empty, so start at cost 0.

→ Do not charge the cell being left. This differs from LC1368.

Problems 9–11: Euler matrix path sums.

→ Entering v costs $\mathrm{matrix}[v]$.

→ The initial cell is counted too: $\mathrm{dist}[s] = \mathrm{matrix}[s]$, not 0.

Movement rules change the graph

Problems 9–11: positive matrix entries; destination-entry costs.

Problem	Moves	Start / finish	Method
9: Euler 81	right, down	top left / bottom right	DP or Dijkstra
10: Euler 82	up, down, right	any left / any right	Dijkstra
11: Euler 83	four directions	top left / bottom right	Dijkstra

For Euler 82, seed the heap with **every left-column cell**, each at its own entry value. Answer: minimum distance in the right column.

For Euler 81, right/down moves let a simple row-by-row DP compute each cell from its upper and left neighbours.

Problem 8: Wizard in Maze

ABC176D has two kinds of moves:

- → Walk to an orthogonally adjacent **open** cell: cost 0.
- → Warp to an **open** cell within a 5×5 square centred on the current cell: cost 1.
- → For a warp, both row and column offsets may be at most 2 in absolute value. Intermediate walls do not matter.

Minimise the number of warps with **0–1 BFS**, not the number of steps.

Check the landing cell. A warp does *not* allow landing on a wall.

Problem 4: maximise a product, not minimise a sum

LC1514: Path with Maximum Probability.

$$P(s \rightsquigarrow t) = \prod_{e \text{ on the path}} p_e, \qquad 0 \le p_e \le 1.$$

- → Start with $\mathrm{best}[s] = 1$; other probabilities are 0.
- → Candidate through u: $\mathrm{best}[u] \cdot p(u, v)$. Update when this is **larger**.
- → Use a **max-heap**; Python's min-heap can store negative probabilities. Skip stale snapshots as before.

Extending a route cannot increase its probability. Therefore the current *maximum* pop is final; the target may be returned then. Return 0 if no route exists.

Problem 5: minimise the largest step

LC1631: Path with Minimum Effort.

$$\text{effort}(P) = \max_{(u,v) \text{ on } P} |h[u] - h[v]|.$$

- → Start at effort 0. This is not a sum of height differences.
- → Relax with $\text{candidate} = \max(\text{effort}[u], |h[u] - h[v]|)$.
- → Keep smaller candidates and use a **min-heap**.

Extending a route cannot reduce its largest step. The same current-minimum settling rule applies.

Pattern: define how a path score extends, then justify why the best current pop cannot improve later.

Try yourself C5: what are we optimising?

These are three **separate** mini-problems.

1. Euler 81 rules on $\begin{pmatrix} 2 & 8 \\ 1 & 3 \end{pmatrix}$: find the minimum path sum, including the starting cell.
2. A direct route has success probability 0.75. A two-edge route has probabilities 0.8 and 0.9. Which route is better?
3. Route A has step differences $2, 9$; route B has $6, 6$. Which has smaller *effort*?

For each, state the operation used to extend a path score.

A constraint may require a bigger state

Problem 12: LC1928, minimum fee within a time limit.

At the same city u, we might arrive:

- $\rightarrow$ cheaply but slowly: fee 3, time used 9;
- $\rightarrow$ expensively but quickly: fee 6, time used 3.

Neither arrival is automatically better: future feasibility depends on the **time remaining**.

Keep a state such as $(\mathrm{city}, \mathrm{timeUsed})$, with the best fee for each state. One `dist[city]` can discard a necessary route.

The source fee is counted; travelling adds time and the destination's fee.

Try yourself C6: keep the useful arrival

Continue the previous example:

- → Total time limit: 10.
- → The only move from u to the target takes 3 more time units.
- → The target's entry fee is 2.

1. Which arrival at u can finish? What is its total fee?
2. Why does retaining only the cheapest fee at u fail?
3. How would you label the states and reject illegal transitions?

Choose by the model, not the problem's title

Need / edge costs	First choice	Time
Reachability only	DFS or BFS	$O(V + E)$
Unit / equal positive cost	BFS	$O(V + E)$
Costs 0 or 1	0–1 BFS	$O(V + E)$
General nonnegative costs	Dijkstra	heap logarithm
General negative costs	e.g. Bellman–Ford	outside today's scope

Multiple starts? Seed **all sources** at their correct initial scores.

Time, fuel, keys or remaining moves affect future choices? They may belong in the **state**, not just in an edge weight.

Problem sheet: live examples 1–3

1. **GFG: Dijkstra Algorithm**
   Undirected edge list; return every distance.
2. **LC743: Network Delay Time**
   Directed edges; return the maximum distance, or -1.
3. **LC1368: Minimum Cost to Make at Least One Valid Path in a Grid**
   Arrow-departure costs; 0–1 BFS.

Use the worked traces to explain each improvement, not just the final answer.

Problem sheet: core practice 4–11

No.	Problem	Modelling cue
4	LC1514: Maximum Probability	product; max-heap
5	LC1631: Minimum Effort	largest step; min-heap
6	LC2290: Obstacle Removal	destination-entry 0/1 cost
7	LC542: 01 Matrix	multi-source *ordinary* BFS
8	ABC176D: Wizard in Maze	free steps, paid warps
9	Euler 81: Path Sum, Two Ways	right/down; DP also works
10	Euler 82: Path Sum, Three Ways	left column as many starts
11	Euler 83: Path Sum, Four Ways	four-direction Dijkstra

Work individually or in small groups. For each problem, first write down **state, legal moves, move cost and initial score**.

Problem sheet: extended challenges 12–15

12. **LC1928: Minimum Cost to Reach Destination in Time**
    State expansion: city alone is not enough.
13. **CF786B: Legacy**
    Segment-tree graph construction plus Dijkstra.
14. **CF715B: Complete The Graph**
    Construct edge weights to meet a shortest-path target.
15. **ABC213E: Stronger Takahashi**
    Advanced 0–1 movement modelling; read the punch rules carefully.

Optional stretch work after the core set; these require additional ideas.

Take away the ordering, not just the code

- → **Model first**: state, moves, costs and starting score.
- → **Dijkstra**: settle the current minimum from a min-heap.
- → **0–1 BFS**: cost 0 to the front, cost 1 to the back.
- → **Both**: allow improvements, skip stale entries, and only early-exit on a current target pop.

Ask: *What guarantees that the next state I process is already optimal?*

End of main slides. Checkpoint answers follow in the appendix.

Answer C1: one queue, two wavefronts

$$\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

- → Seed $(0, 2)$ and $(2, 0)$ at distance 0 **before** expanding.
- → Layer 1: $(0, 1), (1, 2), (1, 0), (2, 1)$.
- → Layer 2: $(0, 0), (1, 1), (2, 2)$.
- → FIFO processing explores all shorter routes first, so first discovery is minimal. DFS does not maintain distance layers and can find a longer route before a shorter one.

Answer C2: zero edges are still safe

With D–E changed to 0:

$$[\mathrm{dist}[A], \mathrm{dist}[B], \mathrm{dist}[C], \mathrm{dist}[D], \mathrm{dist}[E]] = [0, 3, 1, 4, 4].$$

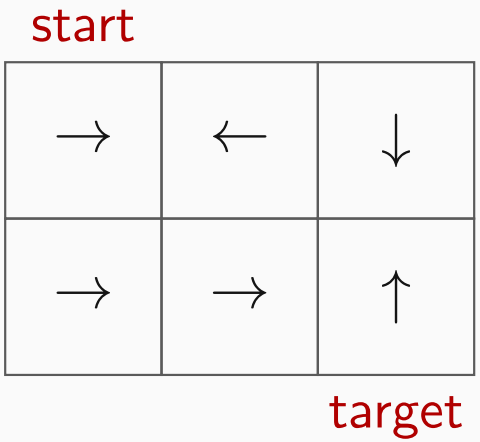
- → Route $A \to C \to B \to D \to E$ costs $1 + 2 + 1 + 0 = 4$.
- → E is first discovered from B with cost $3 + 9 = 12$. This is not final.
- → Current pops are $(0, A), (1, C), (3, B), (4, D), (4, E)$. Equal keys use the alphabetical tie rule stated in the question.
- → Stop safely at $(4, E)$. If continuing, old snapshots $(6, D), (8, B), (12, E)$ are skipped.

Answer C3: discovery is not settlement

1. With enqueue-time visited flags, S fixes A at 1 and B at 0. B cannot improve A, and discovers T at 1. A cannot improve T either. The wrong answer is **1**, instead of 0.
2. Without the stale check, old $(1, A)$ rescans A's outgoing edge, and old $(1, T)$ processes T again. Strict relaxation prevents new improvements here, but these scans are unnecessary.
3. With $\leq$, an edge on a zero-cost cycle can requeue a vertex at the *same* distance forever. Strict $<$ queues only genuine improvements.

The correct shortest route is $S \to B \to A \to T$, total cost 0.

Answer C4: exactly one changed arrow



Zero-cost reachable cells are just $(0, 0)$ and $(0, 1)$: their arrows point to each other.
Any route to the target must leave this set, so it costs **at least 1**.

Route $(0, 0) \to (0, 1) \to (0, 2) \to (1, 2)$ has costs $0, 1, 0$. Change $(0, 1)$ from left to right. Total $\boxed{1}$ matches the lower bound.

Alternatively, change $(0, 0)$ to down, then follow the bottom row.

Answer C4: a reproducible grid-deque trace

Name cells $\begin{smallmatrix} A & B & C \\ D & E & F \end{smallmatrix}$. Scan directions **right, left, down, up**; ignore out-of-bounds moves. Entries are **(stored cost, cell)**, front on the left.

Pop	Improved labels	Deque after the pop
$(0, A)$	B: 0, D: 1	$[(0, B), (1, D)]$
$(0, B)$	C: 1, E: 1	$[(1, D), (1, C), (1, E)]$
$(1, D)$	none	$[(1, C), (1, E)]$
$(1, C)$	F: 1	$[(1, F), (1, E)]$
$(1, F)$	none	$[(1, E)]$
$(1, E)$	none	empty

Complete distances: $\begin{pmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}$. For F alone, stop at its current pop $(1, F)$. The zero cycle A–B creates no new entries because updates are strict.

Answer C5: sum, product and maximum

1. **Sum:** start at 2, then enter cells of value 1 and 3. Going down then right costs $2 + 1 + 3 = \boxed{6}$. Right then down costs $2 + 8 + 3 = 13$. Extend by *adding the destination's value*.
2. **Product:** the two-edge route has probability $0.8 \cdot 0.9 = 0.72$. Choose the direct route, $\boxed{0.75}$. Extend by *multiplying*; maximise with a max-heap.
3. **Maximum:** A's effort is $\max(2, 9) = 9$; B's is $\max(6, 6) = \boxed{6}$. Choose B, despite its larger sum. Extend by *taking a maximum*; minimise with a min-heap.

Answer C6: keep city and time together

- → The cheap arrival would finish at time $9 + 3 = 12 > 10$: **infeasible**.
- → The faster arrival finishes at $3 + 3 = 6 \leq 10$, with total fee $6 + 2 = \boxed{8}$.
- → Keeping only fee 3 at city u discards the arrival that can finish.
- → Let $\mathrm{dist}[u][t]$ be the minimum fee to reach u at time t. A move taking τ tries state $(v, t + \tau)$ with candidate fee $\mathrm{dist}[u][t] + \mathrm{fee}[v]$.

Reject $t + \tau > 10$. Initially $\mathrm{dist}[s][0] = \mathrm{fee}[s]$. Minimise over feasible arrival times at the target.

More advanced solutions may keep only nondominated time–fee pairs; the full state table is the safer starting model.